

Mathematics Foundations of Expectation-Maximization for Hidden Markov Models

EM, Gaussian Mixture Models,
Hidden Markov Models & Baum–Welch

Project Reference

Expectation-Maximisation · EM for GMMs · Forward–Backward · Viterbi ·
Baum–Welch

Contents

1	The Expectation-Maximisation Algorithm	3
1.1	Setup and Motivation	3
1.2	The Evidence Lower Bound (ELBO)	3
1.3	When is the ELBO Tight?	4
1.4	The EM Algorithm	4
2	EM for Gaussian Mixture Models	6
2.1	Model Definition	6
2.2	E-Step: Responsibilities	6
2.3	M-Step: Maximising the ELBO	7
2.3.1	Updating α_j	7
2.3.2	Updating μ_j	7
2.3.3	Updating Σ_j	7
2.4	Full GMM EM Algorithm	8
3	Markov Models and Hidden Markov Models	9
3.1	Markov Chains	9
3.2	Hidden Markov Models (HMMs)	9
3.3	HMM Parameters	10
3.4	Three Inference Problems	10
4	The Forward–Backward Algorithm	11
4.1	Motivation	11
4.2	Forward Algorithm	11
4.3	Backward Algorithm	12
4.4	Combining Forward and Backward	12
4.5	Numerical Stability: The Log-Sum-Exp Trick	12
5	The Viterbi Algorithm	14
5.1	Problem: Most Probable Hidden Sequence	14
5.2	Viterbi Variable	14
6	Baum–Welch: EM for HMMs	15
6.1	Setup	15
6.1.1	Complete Data Log-Likelihood	15
6.2	E-Step: Computing the Sufficient Posteriors	15
6.3	The ELBO for HMMs	16
6.4	M-Step: Parameter Updates	16
6.4.1	Updating π	16

6.4.2	Updating $T_{i,j}$	17
6.4.3	Updating ξ_i (Discrete Emissions)	17
6.4.4	Updating ξ_i (Gaussian Emissions)	18
6.5	Full Baum–Welch Algorithm	18
7	Unified Summary and Correspondence Table	20

1. The Expectation-Maximisation Algorithm

1.1 Setup and Motivation

Let θ denote model parameters and let $x_1, \dots, x_n$ be n realisations of n identically distributed random variables. We want to maximise the **log-likelihood**:

$$\ell(\theta) = \sum_{i=1}^n \log P_{\theta}(x_i).$$

For models with **latent variables** z , the marginal likelihood involves a sum/integral over z that is generally intractable:

$$P_{\theta}(x_i) = \sum_z P_{\theta}(x_i, z) \quad (\text{discrete } z).$$

Remark Minimising KL $\Leftrightarrow$ Maximising Likelihood

Minimising the KL divergence between the model and the data distribution is equivalent to maximising the likelihood.

1.2 The Evidence Lower Bound (ELBO)

Introduce any density $q(z)$ over the latent variable. For a single example:

Derivation: Lower-bounding $\log P_{\theta}(x_i)$

$$\begin{aligned} \ell(\theta) &= \log P_{\theta}(x_i) \\ &= \log \sum_z P_{\theta}(x_i, z) \\ &= \log \sum_z q(z) \frac{P_{\theta}(x_i, z)}{q(z)} \\ &= \log \mathbb{E}_{q(z)} \left[\frac{P_{\theta}(x_i, z)}{q(z)} \right] \\ &\geq \mathbb{E}_{q(z)} \left[\log \frac{P_{\theta}(x_i, z)}{q(z)} \right] \quad (\text{Jensen's inequality, log is concave}). \end{aligned} \tag{1.1}$$

Definition Evidence Lower Bound

The **ELBO** (Evidence Lower Bound) is defined as:

$$\mathcal{F}_\theta(q) = \mathbb{E}_{q(z)} \left[\log \frac{P_\theta(x, z)}{q(z)} \right] = \mathbb{E}_{q(z)} [\log P_\theta(x, z)] - \mathbb{E}_{q(z)} [\log q(z)].$$

It satisfies $\mathcal{F}_\theta(q) \leq \ell(\theta)$ for all q .

1.3 When is the ELBO Tight?**Derivation: Gap between $\ell(\theta)$ and the ELBO**

$$\begin{aligned} \ell(\theta) - \mathcal{F}_\theta(q) &= \log P_\theta(x) - \mathbb{E}_{q(z)} \left[\log \frac{P_\theta(x, z)}{q(z)} \right] \\ &= \mathbb{E}_{q(z)} [\log P_\theta(x)] - \mathbb{E}_{q(z)} [\log P_\theta(x, z)] + \mathbb{E}_{q(z)} [\log q(z)] \\ &= \mathbb{E}_{q(z)} \left[\log \frac{q(z)}{P_\theta(z | x)} \right] \\ &= D_{\text{KL}}(q(z) \parallel P_\theta(z | x)) \geq 0. \end{aligned} \tag{1.2}$$

Key Result Tight ELBO

The ELBO equals $\ell(\theta)$ (i.e. the KL gap vanishes) if and only if

$$q^*(z) = P_\theta(z | x).$$

1.4 The EM Algorithm

The EM algorithm alternates between tightening the ELBO and maximising it:

Algorithm:

Expectation-Maximisation **Initialise** θ^0 .

For $t = 1, 2, \dots$ until convergence **do**:

E-Step (make ELBO tight at θ^t):

$$q^t(z) \leftarrow P_{\theta^t}(z | x)$$

This sets $\mathcal{F}_{\theta^t}(q^t) = \ell(\theta^t)$.

M-Step (maximise ELBO over θ):

$$\theta^{t+1} \leftarrow \operatorname{argmax}_{\theta} \mathcal{F}_\theta(q^t) = \operatorname{argmax}_{\theta} \mathbb{E}_{q^t(z)} [\log P_\theta(x, z)]$$

(The entropy term $-\mathbb{E}_{q^t} [\log q^t]$ does not depend on θ .)

Remark

The M-step objective $\mathbb{E}_{q^t}[\log P_\theta(x, z)]$ replaces the intractable $\log \sum_z P_\theta(x, z)$ with a *sum* of logs — a crucial simplification.

2. EM for Gaussian Mixture Models

2.1 Model Definition

Definition Gaussian Mixture Model

A **GMM** with m components defines:

$$\begin{aligned} P(z = k) &= \alpha_k, & \alpha_k &\geq 0, & \sum_k \alpha_k &= 1, \\ P(x_i | z = k) &= \mathcal{N}(x_i; \mu_k, \Sigma_k). \end{aligned}$$

The marginal density is:

$$P_\theta(x_i) = \sum_{k=1}^m \alpha_k \mathcal{N}(x_i; \mu_k, \Sigma_k),$$

where $\theta = \{\alpha_k, \mu_k, \Sigma_k\}_{k=1}^m$.

The MLE objective is:

$$\theta^* = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^n \log P_\theta(x_i) = \underset{\theta}{\operatorname{argmax}} \sum_{i=1}^n \log \sum_{k=1}^m \alpha_k \mathcal{N}(x_i; \mu_k, \Sigma_k).$$

Direct optimisation is hard due to the *log of a sum*.

2.2 E-Step: Responsibilities

Setting $q^t(z) = P_{\theta^t}(z | x_i)$, the posterior responsibility of component j for data point x_i is:

Key Result E-Step for GMM

$$r_{ij} = P_{\theta^t}(z = j | x_i) = \frac{\alpha_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}{\sum_{k=1}^m \alpha_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}.$$

2.3 M-Step: Maximising the ELBO

The ELBO for the full dataset, substituting the GMM joint:

$$\begin{aligned}\mathcal{F}_\theta(q^t) &= \mathbb{E}_{q^t}[\log P_\theta(x, z)] \\ &= \sum_{i=1}^n \sum_{j=1}^m r_{ij} \left[\log \mathcal{N}(x_i; \mu_j, \Sigma_j) + \log \alpha_j \right].\end{aligned}\tag{2.1}$$

2.3.1 Updating α_j

Maximise $\sum_{i,j} r_{ij} \log \alpha_j$ subject to $\sum_j \alpha_j = 1$. Form the Lagrangian:

$$\mathcal{L} = \sum_{i=1}^n \sum_{j=1}^m r_{ij} \log \alpha_j + \lambda \left(\sum_{j=1}^m \alpha_j - 1 \right).$$

Setting $\partial \mathcal{L} / \partial \alpha_j = 0$:

$$\frac{\sum_i r_{ij}}{\alpha_j} + \lambda = 0 \implies \alpha_j \propto \sum_i r_{ij}.$$

Using the normalisation constraint $\sum_j \alpha_j = 1$ and $\sum_j \sum_i r_{ij} = n$:

Key Result

$$\alpha_j^{t+1} = \frac{1}{n} \sum_{i=1}^n r_{ij}.$$

2.3.2 Updating μ_j

Setting $\partial \mathcal{F} / \partial \mu_j = 0$ (using the Gaussian log-likelihood):

$$\sum_{i=1}^n r_{ij} \Sigma_j^{-1} (x_i - \mu_j) = 0 \implies \sum_i r_{ij} x_i - \mu_j \sum_i r_{ij} = 0.$$

Key Result

$$\mu_j^{t+1} = \frac{\sum_{i=1}^n r_{ij} x_i}{\sum_{i=1}^n r_{ij}}.$$

2.3.3 Updating Σ_j

Differentiating with respect to Σ_j and equating to zero:

Key Result

$$\Sigma_j^{t+1} = \frac{\sum_{i=1}^n r_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top}{\sum_{i=1}^n r_{ij}}.$$

2.4 Full GMM EM Algorithm**Algorithm:**

EM for GMMs **Initialise** $\theta^0 = \{\alpha_k^0, \mu_k^0, \Sigma_k^0\}$.

For $t = 1, 2, \dots$ until convergence:

E-Step: Compute responsibilities for all $i \in [n], j \in [m]$:

$$r_{ij} = \frac{\alpha_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}{\sum_{k=1}^m \alpha_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}$$

M-Step: Update parameters:

$$\begin{aligned} \alpha_j^{t+1} &= \frac{1}{n} \sum_i r_{ij} \\ \mu_j^{t+1} &= \frac{\sum_i r_{ij} x_i}{\sum_i r_{ij}} \\ \Sigma_j^{t+1} &= \frac{\sum_i r_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top}{\sum_i r_{ij}} \end{aligned}$$

3. Markov Models and Hidden Markov Models

3.1 Markov Chains

Key idea: “Future is independent of past, given the present.”

Markov models are used for **temporal/sequential data**: weather, finance, language, music, etc. Given data $D = (x_1, x_2, \dots, x_n)$ where $x_1, x_2, \dots, x_n$ are *not* i.i.d. (they have dependencies), x_t depends on $x_{t-1}, x_{t-2}, \dots, x_{t-m}$ for some fixed order m .

Definition Discrete-Time Markov Chain

Discrete random variables $X_1, X_2, \dots, X_n$ (taking values in a discrete state space) form a **(first-order) Markov Chain** if:

$$P(x_t \mid x_1, \dots, x_{t-1}) = P(x_t \mid x_{t-1}) \quad \forall t.$$

This gives the joint factorisation:

$$P(x_1, \dots, x_n) = P(x_1) \prod_{t=2}^n P(x_t \mid x_{t-1}).$$

Remark Continuous-Time Markov Processes

Brownian motion (continuous space) and the **Poisson process** (discrete space) are continuous-time analogues.

3.2 Hidden Markov Models (HMMs)

In practice, we often **cannot** perfectly observe the true state of the system. There is *hidden information* — we need to model with hidden/latent variables.

Definition Hidden Markov Model

An HMM has:

- **Hidden states:** $z_1, \dots, z_n \in \{1, \dots, m\}$ (discrete RVs, latent)
- **Observations:** $x_1, \dots, x_n \in \mathcal{X}$ (discrete, $\mathbb{R}$, or $\mathbb{R}^d$)

The *Trellis diagram* structure is:

$$z_1 \rightarrow z_2 \rightarrow z_3 \rightarrow \dots \rightarrow z_n, \quad x_k \text{ emitted from } z_k.$$

The joint distribution factorises as:

$$P(x_1, \dots, x_n, z_1, \dots, z_n) = P(z_1) P(x_1 \mid z_1) \prod_{k=2}^n P(z_k \mid z_{k-1}) P(x_k \mid z_k).$$

Example: Handwriting recognition — hidden states are letters, observations are pixel images.

3.3 HMM Parameters

Definition HMM Parameter Set $\theta = (T, \xi, \pi)$

1. **Transition matrix:** $T_{i,j} = P(z_{k+1} = j \mid z_k = i)$, $i, j \in \{1, \dots, m\}$. Each row sums to 1.
2. **Emission probabilities/densities:** $\xi_i(x) = P(x_k = x \mid z_k = i)$, $x \in \mathcal{X}$. (ξ_i is a probability density on $\mathcal{X}$.) For discrete x : $\xi_i(x) = P(x_k = x \mid z_k = i)$ is a PMF.
3. **Initial distribution:** $\pi(i) = P(z_1 = i)$, $i \in \{1, \dots, m\}$.

Using these, the full joint is:

$$P(x_{1:n}, z_{1:n}) = \pi(z_1) \xi_{z_1}(x_1) \prod_{k=2}^n T_{z_{k-1}, z_k} \xi_{z_k}(x_k).$$

Example (from notes): State space $z_k \in \{-1, +1\}$, observations $x_k \in \mathbb{R}$, with transition matrix:

$$T = \begin{pmatrix} 0.99 & 0.01 \\ 0.01 & 0.99 \end{pmatrix}.$$

(Rows sum to 1.)

3.4 Three Inference Problems

Given observations $x = x_{1:n}$ and parameters θ :

1. **Posterior inference:** compute $P(z_k \mid x)$ or $P(z_k, z_{k+1} \mid x)$ — done via **Forward–Backward**.
2. **Decoding:** find $z^* = \operatorname{argmax}_z P(z \mid x)$ — done via **Viterbi**.
3. **Parameter learning:** estimate θ — done via **Baum–Welch** (EM for HMMs).

4. The Forward–Backward Algorithm

4.1 Motivation

The naive approach to computing $P(z_k | x)$ is:

$$P(z_k | x) = \frac{P(z_k, x)}{P(x)}, \quad P(x) = \sum_{z_{1:n}} P(x, z_{1:n}) = \sum_{z_1} \sum_{z_2} \cdots \sum_{z_n} P(x, z).$$

The innermost computation scales as $\mathcal{O}(m^n)$ — exponential in sequence length. A dynamic programming recursion reduces this to $\mathcal{O}(nm^2)$.

4.2 Forward Algorithm

Definition Forward Variable

$$\alpha_k(z_k) = P(z_k, x_{1:k}).$$

Derivation: Forward Recursion

$$\begin{aligned} \alpha_k(z_k) &= P(z_k, x_{1:k}) \\ &= \sum_{z_{k-1}} P(z_k, z_{k-1}, x_{1:k}) \\ &= \sum_{z_{k-1}} P(x_k | z_k) P(z_k | z_{k-1}) P(z_{k-1}, x_{1:k-1}) \\ &= \underbrace{P(x_k | z_k)}_{\xi_{z_k}(x_k)} \sum_{z_{k-1}} \underbrace{P(z_k | z_{k-1})}_{T_{z_{k-1}, z_k}} \underbrace{P(z_{k-1}, x_{1:k-1})}_{\alpha_{k-1}(z_{k-1})}. \end{aligned} \tag{4.1}$$

Key Result Forward Recursion

$$\alpha_k(z_k) = \xi_{z_k}(x_k) \sum_{z_{k-1}=1}^m T_{z_{k-1}, z_k} \alpha_{k-1}(z_{k-1}), \quad k = 2, \dots, n.$$

Base case: $\alpha_1(z_1) = P(z_1, x_1) = \pi(z_1) \xi_{z_1}(x_1)$.

Running this for all $k = 1, \dots, n$ and all $z_k \in \{1, \dots, m\}$ gives us $P(z_k, x_{1:k})$ for every k , at a cost of $\mathcal{O}(m)$ per z_k , $\mathcal{O}(m^2)$ per k , and $\mathcal{O}(nm^2)$ total.

4.3 Backward Algorithm

Definition Backward Variable

$$\beta_k(z_k) = P(x_{k+1:n} \mid z_k), \quad k = 1, \dots, n-1.$$

Derivation: Backward Recursion

$$\beta_k(z_k) = P(x_{k+1:n} \mid z_k) \tag{4.2}$$

$$\begin{aligned} &= \sum_{z_{k+1}} P(x_{k+1:n}, z_{k+1} \mid z_k) \\ &= \sum_{z_{k+1}} P(x_{k+2:n} \mid z_{k+1}) P(x_{k+1} \mid z_{k+1}) P(z_{k+1} \mid z_k) \\ &= \sum_{z_{k+1}} \underbrace{\beta_{k+1}(z_{k+1})}_{\text{recursive}} \underbrace{\xi_{z_{k+1}}(x_{k+1})}_{\text{emission}} \underbrace{T_{z_k, z_{k+1}}}_{\text{transition}}. \end{aligned} \tag{4.3}$$

Key Result Backward Recursion

$$\beta_k(z_k) = \sum_{z_{k+1}=1}^m \beta_{k+1}(z_{k+1}) \xi_{z_{k+1}}(x_{k+1}) T_{z_k, z_{k+1}}, \quad k = n-1, \dots, 1.$$

Base case: $\beta_n(z_n) = 1$ for all z_n .

Both Forward and Backward run in $\mathcal{O}(nm^2)$.

4.4 Combining Forward and Backward

Key Result Posterior State Marginal

Using conditional independence of past and future given z_k :

$$P(z_k, x_{1:n}) = P(z_k, x_{1:k}) \cdot P(x_{k+1:n} \mid z_k) = \alpha_k(z_k) \beta_k(z_k).$$

Hence:

$$P(z_k \mid x_{1:n}) = \frac{\alpha_k(z_k) \beta_k(z_k)}{\sum_{j=1}^m \alpha_k(j) \beta_k(j)},$$

where the denominator equals $P(x_{1:n})$.

4.5 Numerical Stability: The Log-Sum-Exp Trick

The forward variable α_k involves products of many probabilities and **underflows** to zero for long sequences. Compute in log-space using:

Key Result Log-Sum-Exp Trick

Given values $a_1, \dots, a_m$, let $b = \max_i a_i$. Then:

$$\log \sum_{i=1}^m e^{a_i} = b + \log \sum_{i=1}^m e^{a_i - b}.$$

Applying to the forward recursion: let $a_i = \log P(x_k \mid z_k) + \log T_{z_{k-1}, z_k} + \log \alpha_{k-1}(z_{k-1})$. Then:

$$\log \alpha_k(z_k) = \log \sum_{z_{k-1}} \exp[\log \xi_{z_k}(x_k) + \log T_{z_{k-1}, z_k} + \log \alpha_{k-1}(z_{k-1})].$$

5. The Viterbi Algorithm

5.1 Problem: Most Probable Hidden Sequence

Given observations $x = x_{1:n}$, find the most probable hidden state sequence:

$$z^* = \operatorname{argmax}_{z_{1:n}} P(z_{1:n} \mid x_{1:n}) = \operatorname{argmax}_{z_{1:n}} P(z_{1:n}, x_{1:n}).$$

5.2 Viterbi Variable

Definition Viterbi Variable

$$\mu_k(z_k) = \max_{z_{1:k-1}} P(z_{1:k}, x_{1:k}).$$

Derivation: Viterbi Recursion

Using the key identity: if $f(a) \geq 0$ for all a and $g(a, b) \geq 0$ for all a, b , then: $\max_{a,b} f(a) g(a, b) = \max_a [f(a) \max_b g(a, b)]$.

$$\begin{aligned} \mu_k(z_k) &= \max_{z_{1:k-1}} P(z_{1:k}, x_{1:k}) \\ &= \max_{z_{k-1}} P(x_k \mid z_k) P(z_k \mid z_{k-1}) \max_{z_{1:k-2}} P(z_{1:k-1}, x_{1:k-1}) \\ &= P(x_k \mid z_k) \max_{z_{k-1}} \left[P(z_k \mid z_{k-1}) \mu_{k-1}(z_{k-1}) \right]. \end{aligned} \tag{5.1}$$

Key Result Viterbi Recursion

$$\mu_k(z_k) = \xi_{z_k}(x_k) \max_{z_{k-1}} \left[T_{z_{k-1}, z_k} \mu_{k-1}(z_{k-1}) \right], \quad k = 2, \dots, n.$$

Base case: $\mu_1(z_1) = P(z_1, x_1) = \pi(z_1) \xi_{z_1}(x_1)$.

The optimal sequence is recovered by: $z_n^* = \operatorname{argmax}_{z_n} \mu_n(z_n)$, then backtracking.

Remark Viterbi vs. Forward-Backward

Viterbi uses max where Forward uses $\sum$. Both have complexity $\mathcal{O}(nm^2)$.

6. Baum–Welch: EM for HMMs

6.1 Setup

We now apply the EM framework to learn HMM parameters $\theta = (T, \xi, \pi)$ from observed data $x = x_{1:n}$, treating the hidden states $z = z_{1:n}$ as latent variables.

6.1.1 Complete Data Log-Likelihood

Using the HMM joint factorisation:

$$\begin{aligned} \log P_\theta(x_{1:n}, z_{1:n}) &= \log \pi(z_1) + \log \xi_{z_1}(x_1) \\ &\quad + \sum_{k=2}^n \left[\log T_{z_{k-1}, z_k} + \log \xi_{z_k}(x_k) \right]. \end{aligned} \quad (6.1)$$

6.2 E-Step: Computing the Sufficient Posteriors

Setting $q^t(z) = P_{\theta^t}(z_{1:n} \mid x_{1:n})$ makes the ELBO tight. Rather than storing the full joint (cost: m^n), we extract two marginals via Forward–Backward.

Definition Smoothed State Posterior $\gamma_k(i)$

$$\gamma_k(i) = P(z_k = i \mid x_{1:n}).$$

Key Result

$$\gamma_k(i) = \frac{\alpha_k(i) \beta_k(i)}{\sum_{j=1}^m \alpha_k(j) \beta_k(j)}.$$

Note: the denominator $= P(x_{1:n})$, independent of k — a useful check.

Definition Pairwise Transition Posterior $\varphi_k(i, j)$

$$\varphi_k(i, j) = P(z_k = i, z_{k+1} = j \mid x_{1:n}), \quad k = 1, \dots, n-1.$$

Derivation: Computing $\varphi_k(i, j)$

Split the sequence at the transition $z_k \rightarrow z_{k+1}$:

$$P(z_k = i, z_{k+1} = j, x_{1:n}) = \underbrace{P(z_k = i, x_{1:k})}_{\alpha_k(i)} \cdot \underbrace{T_{i,j}}_{\text{transition}} \cdot \underbrace{\xi_j(x_{k+1})}_{\text{emission}} \cdot \underbrace{P(x_{k+2:n} | z_{k+1} = j)}_{\beta_{k+1}(j)}. \quad (6.2)$$

Normalising:

Key Result Pairwise Posterior

$$\varphi_k(i, j) = \frac{\alpha_k(i) T_{i,j} \xi_j(x_{k+1}) \beta_{k+1}(j)}{\sum_{i'=1}^m \sum_{j'=1}^m \alpha_k(i') T_{i',j'} \xi_{j'}(x_{k+1}) \beta_{k+1}(j')}.$$

Remark Consistency

Marginalising φ_k over j recovers γ_k :

$$\sum_{j=1}^m \varphi_k(i, j) = \gamma_k(i) \quad \forall k \in \{1, \dots, n-1\}.$$

This is the HMM analogue of $\sum_j r_{ij} = 1$ in GMMs.

6.3 The ELBO for HMMS

Substituting the HMM complete data log-likelihood into the ELBO $\mathcal{F}_\theta(q^t) = \mathbb{E}_{q^t}[\log P_\theta(x, z)]$:

Key Result HMM ELBO

$$\mathcal{F}_\theta(q^t) = \underbrace{\sum_{i=1}^m \gamma_1(i) \log \pi(i)}_{\text{initial term}} + \underbrace{\sum_{k=1}^{n-1} \sum_{i=1}^m \sum_{j=1}^m \varphi_k(i, j) \log T_{i,j}}_{\text{transition term}} + \underbrace{\sum_{k=1}^n \sum_{i=1}^m \gamma_k(i) \log \xi_i(x_k)}_{\text{emission term}}. \quad (6.3)$$

The three terms **decouple completely**, so π , T , and ξ can each be optimised independently.

6.4 M-Step: Parameter Updates**6.4.1 Updating π**

The initial term $\sum_i \gamma_1(i) \log \pi(i)$ is maximised directly (only one time step contributes):

Key Result

$$\pi^{t+1}(i) = \gamma_1(i).$$

6.4.2 Updating $T_{i,j}$

Maximise $\sum_{k,i,j} \varphi_k(i,j) \log T_{i,j}$ subject to $\sum_j T_{i,j} = 1$ for each i .

Form the Lagrangian with multiplier λ_i per row:

$$\mathcal{L} = \sum_{k=1}^{n-1} \sum_{i,j} \varphi_k(i,j) \log T_{i,j} + \sum_i \lambda_i \left(\sum_j T_{i,j} - 1 \right).$$

Setting $\partial \mathcal{L} / \partial T_{i,j} = 0$:

$$\frac{\sum_{k=1}^{n-1} \varphi_k(i,j)}{T_{i,j}} + \lambda_i = 0 \implies T_{i,j} \propto \sum_{k=1}^{n-1} \varphi_k(i,j).$$

Normalising using $\sum_j T_{i,j} = 1$ and $\sum_j \sum_k \varphi_k(i,j) = \sum_k \gamma_k(i)$:

Key Result

$$T_{i,j}^{t+1} = \frac{\sum_{k=1}^{n-1} \varphi_k(i,j)}{\sum_{k=1}^{n-1} \gamma_k(i)}.$$

Interpretation: expected number of $i \rightarrow j$ transitions divided by expected time spent in state i .

Remark Analogy with GMM

Compare with the GMM update $\alpha_j^{t+1} = \frac{1}{n} \sum_i r_{ij}$: the structure is identical, with $\varphi_k(i,j)$ playing the role of r_{ij} and the denominator normalising over all transitions out of state i .

6.4.3 Updating ξ_i (Discrete Emissions)

Maximise $\sum_{k,i} \gamma_k(i) \log \xi_i(x_k)$ over the emission PMF $\xi_i(\cdot)$ subject to $\sum_x \xi_i(x) = 1$. By the same Lagrangian argument:

Key Result

$$\xi_i^{t+1}(x) = \frac{\sum_{k=1}^n \gamma_k(i) \mathbf{1}[x_k = x]}{\sum_{k=1}^n \gamma_k(i)}.$$

Interpretation: fraction of time the system is in state i and emits symbol x .

6.4.4 Updating ξ_i (Gaussian Emissions)

For $\xi_i = \mathcal{N}(\mu_i, \Sigma_i)$, the emission term becomes $\sum_{k,i} \gamma_k(i) \log \mathcal{N}(x_k; \mu_i, \Sigma_i)$. Setting gradients to zero — **identical in form** to the GMM M-step with $r_{ij} \rightarrow \gamma_k(i)$:

Key Result

$$\mu_i^{t+1} = \frac{\sum_{k=1}^n \gamma_k(i) x_k}{\sum_{k=1}^n \gamma_k(i)}, \quad (6.4)$$

$$\Sigma_i^{t+1} = \frac{\sum_{k=1}^n \gamma_k(i) (x_k - \mu_i)(x_k - \mu_i)^\top}{\sum_{k=1}^n \gamma_k(i)}. \quad (6.5)$$

6.5 Full Baum–Welch Algorithm**Algorithm:**

Baum–Welch (EM for HMMs) **Initialise** $\theta^0 = (T^0, \xi^0, \pi^0)$.

For $t = 1, 2, \dots$ until convergence:

E-Step:

1. Run **Forward Algorithm**: $\alpha_1(z_1) = \pi(z_1) \xi_{z_1}(x_1)$, then recurse to get $\alpha_k(z_k)$ for all k .
2. Run **Backward Algorithm**: $\beta_n(z_n) = 1$, then recurse backwards to get $\beta_k(z_k)$.
3. Compute smoothed posteriors:

$$\gamma_k(i) = \frac{\alpha_k(i) \beta_k(i)}{\sum_j \alpha_k(j) \beta_k(j)}$$

4. Compute pairwise posteriors:

$$\varphi_k(i, j) = \frac{\alpha_k(i) T_{i,j} \xi_j(x_{k+1}) \beta_{k+1}(j)}{P(x_{1:n})}$$

M-Step:

$$\begin{aligned} \pi^{t+1}(i) &= \gamma_1(i) \\ T_{i,j}^{t+1} &= \frac{\sum_{k=1}^{n-1} \varphi_k(i, j)}{\sum_{k=1}^{n-1} \gamma_k(i)} \\ \xi_i^{t+1}(x) &= \frac{\sum_k \gamma_k(i) \mathbf{1}[x_k = x]}{\sum_k \gamma_k(i)} \quad (\text{discrete}) \end{aligned}$$

Remark Complexity

Each iteration costs $\mathcal{O}(nm^2)$ — dominated by the Forward–Backward passes. Compare with the naïve $\mathcal{O}(m^n)$ for the full posterior.

7. Unified Summary and Correspondence Table

The table below makes the deep structural correspondence between EM for GMMs and Baum–Welch explicit.

EM for GMMs	Baum–Welch (EM for HMMs)
Latent variable $z \in \{1, \dots, m\}$	Hidden sequence $z_{1:n}$, each $z_k \in \{1, \dots, m\}$
Responsibility: $r_{ij} = P_\theta(z = j \mid x_i)$	Smoothed posterior: $\gamma_k(i) = P_\theta(z_k = i \mid x_{1:n})$
—	Pairwise posterior: $\varphi_k(i, j) = P_\theta(z_k = i, z_{k+1} = j \mid x_{1:n})$
E-step: Bayes’ rule	E-step: Forward–Backward algorithm
M-step: closed-form updates	M-step: closed-form updates
$\alpha_j^{t+1} = \frac{1}{n} \sum_i r_{ij}$	$\pi^{t+1}(i) = \gamma_1(i)$
$\mu_j^{t+1} = \frac{\sum_i r_{ij} x_i}{\sum_i r_{ij}}$	$\mu_i^{t+1} = \frac{\sum_k \gamma_k(i) x_k}{\sum_k \gamma_k(i)}$
$\Sigma_j^{t+1} = \frac{\sum_i r_{ij} (x_i - \mu_j)(x_i - \mu_j)^\top}{\sum_i r_{ij}}$	$\Sigma_i^{t+1} = \frac{\sum_k \gamma_k(i) (x_k - \mu_i)(x_k - \mu_i)^\top}{\sum_k \gamma_k(i)}$
No transition structure	$T_{i,j}^{t+1} = \frac{\sum_k \varphi_k(i, j)}{\sum_k \gamma_k(i)}$
$\mathcal{O}(nm)$ per iteration	$\mathcal{O}(nm^2)$ per iteration

The key structural difference is that hidden states in an HMM are **sequentially dependent**, which (1) prevents us from collapsing the E-step to a simple Bayes’ rule, requiring Forward–Backward instead, and (2) introduces the transition matrix T as an additional parameter block with its own M-step equation driven by φ_k .

Once $\gamma_k(i)$ and $\varphi_k(i, j)$ are in hand, the M-step updates for μ , Σ are **structurally identical** to GMM EM — $\gamma_k(i)$ plays the exact role of the GMM responsibility r_{ij} .